



SESSION 20

Eigendecomposition & PCA

Matrix Diagonalization · Spectral Theorem · PCA Derivation · Kernel & Incremental PCA

Matrix Diagonalization & Eigendecomposition

Symmetric Matrices & Spectral Theorem

PCA Problem Formulation

Covariance Matrix Derivation

PCA Algorithm Steps

Reconstruction Error & Variance Explained

PCA Variants (Kernel & Incremental)

Python PCA Implementation

Session Notes — Complete Reference

Playlist: Maths for Machine Learning · Instructor: Nitish Singh (CampusX)



Table of Contents

01 Matrix Diagonalization & Eigendecomposition

02 Symmetric Matrices & Spectral Theorem

03 PCA Problem Formulation

04 Covariance Matrix Derivation

05 PCA Algorithm Steps

06 Reconstruction Error & Variance Explained

07 PCA Variants (Kernel & Incremental)

08 Python PCA Implementation

09 Quick Reference Card

01 • Matrix Diagonalization & Eigendecomposition

If an $n \times n$ matrix A has n linearly independent eigenvectors, it can be factored into a product of matrices:

EIGENDECOMPOSITION

$A = P \cdot D \cdot P^{-1}$ where P 's columns are eigenvectors and D is diagonal with eigenvalues.

02 · Symmetric Matrices & Spectral Theorem

The **Spectral Theorem** states that any real symmetric matrix A can be orthogonally diagonalized:

ORTHOGONAL DIAGONALIZATION

$$A = Q \cdot D \cdot Q^T \text{ (since } Q^T = Q^{-1} \text{)}$$

03 · PCA Problem Formulation

Principal Component Analysis (PCA) is a dimensionality reduction technique. The goal is to project data onto a lower-dimensional subspace while maximizing the variance of the projected points.

04 · Covariance Matrix Derivation

PCA computes the eigenvectors of the **covariance matrix** Σ of the centered data matrix X :

COVARIANCE MATRIX

$$\Sigma = (1 / n) \cdot X^T X$$

The eigenvectors of Σ represent the principal directions of variance, and the eigenvalues denote the magnitude of variance in those directions.

05 • PCA Algorithm Steps

1. Standardize (center and scale) the dataset.
2. Compute the covariance matrix Σ .
3. Compute eigenvalues and eigenvectors of Σ .
4. Sort eigenvalues in descending order.
5. Select the top k eigenvectors to form the projection matrix W .
6. Transform the original data: $Y = XW$.

06 · Reconstruction Error & Variance Explained

The proportion of variance explained by the top k principal components is:

EXPLAINED VARIANCE RATIO

$$\text{Explained Variance} = \sum_{i=1}^k \lambda_i / \sum_{j=1}^d \lambda_j$$

07 · PCA Variants (Kernel & Incremental)

- **Kernel PCA:** Uses the kernel trick to perform non-linear dimensionality reduction.
- **Incremental PCA:** Processes data in mini-batches, suitable for large datasets that do not fit in RAM.

08 • Python PCA Implementation

```
from sklearn.decomposition import PCA
import numpy as np

X = np.random.randn(100, 5)

pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)

print("Explained Variance Ratio:", pca.explained_variance_ratio_)
```

09 · Quick Reference Card



PCA & Eigendecomposition Cheat Sheet

- **Eigendecomposition:** $A = PDP^{-1}$.
- **PCA objective:** Maximize projected variance / minimize reconstruction error.
- **Scree Plot:** Plot of eigenvalues to help choose the number of principal components.